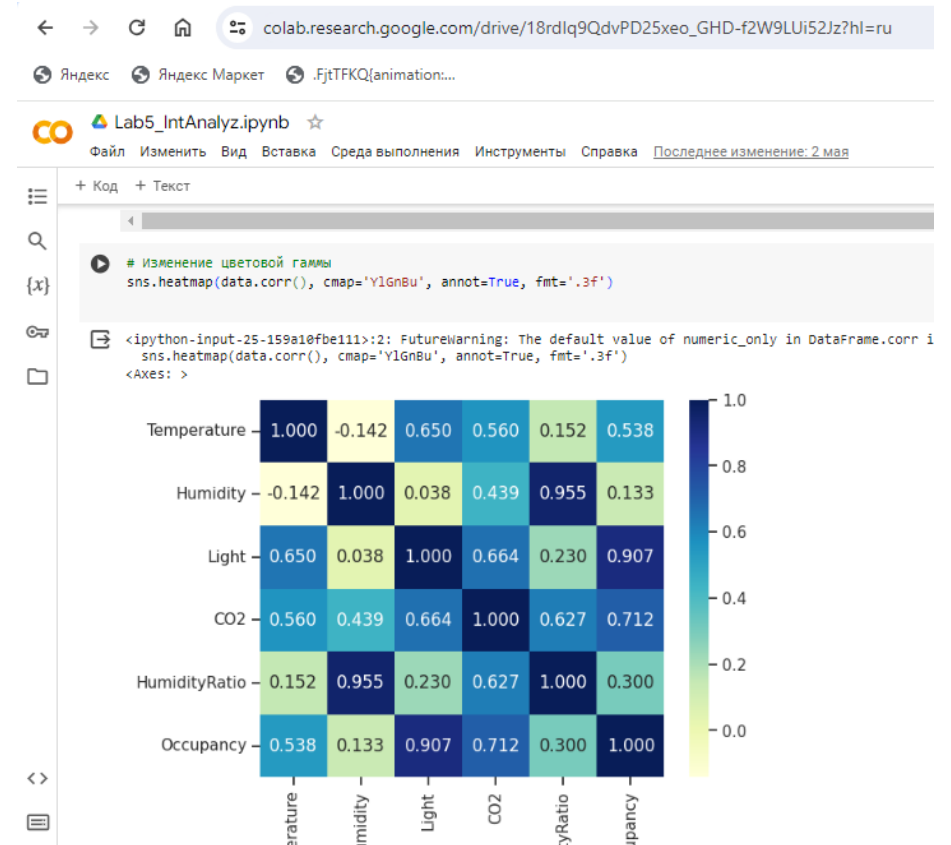


# Введение в машинное обучение

Быстренина Ирина Евгеньевна, к.п.н., доцент  
[Bystrenina.IE@rea.ru](mailto:Bystrenina.IE@rea.ru)

**Python** — один из наиболее популярных языков программирования для машинного обучения.

**Google Colab** — сервис, созданный Google, который предоставляет возможность работать с кодом на языке Python через Jupyter Notebook, не устанавливая на свой компьютер дополнительных программ. В Google Colab можно применять различные библиотеки на Python, загружать и запускать файлы, анализировать данные и получать результаты в браузере.



Библиотеки машинного обучения — это наборы инструментов и алгоритмов, которые позволяют создавать и обучать модели машинного обучения. Некоторые из наиболее популярных библиотек машинного обучения включают TensorFlow, PyTorch, Scikit-learn, Pandas, Numpy, Matplotlib, Seaborn и др.

# НАЗНАЧЕНИЕ БИБЛИОТЕКИ SK-LEARN

**Scikit-learn** - один из наиболее широко используемых пакетов Python для Data Science и Machine Learning. Он позволяет выполнять множество операций и предоставляет множество алгоритмов.

Scikit-learn также предлагает отличную документацию о своих классах, методах и функциях, а также описание используемых алгоритмов.

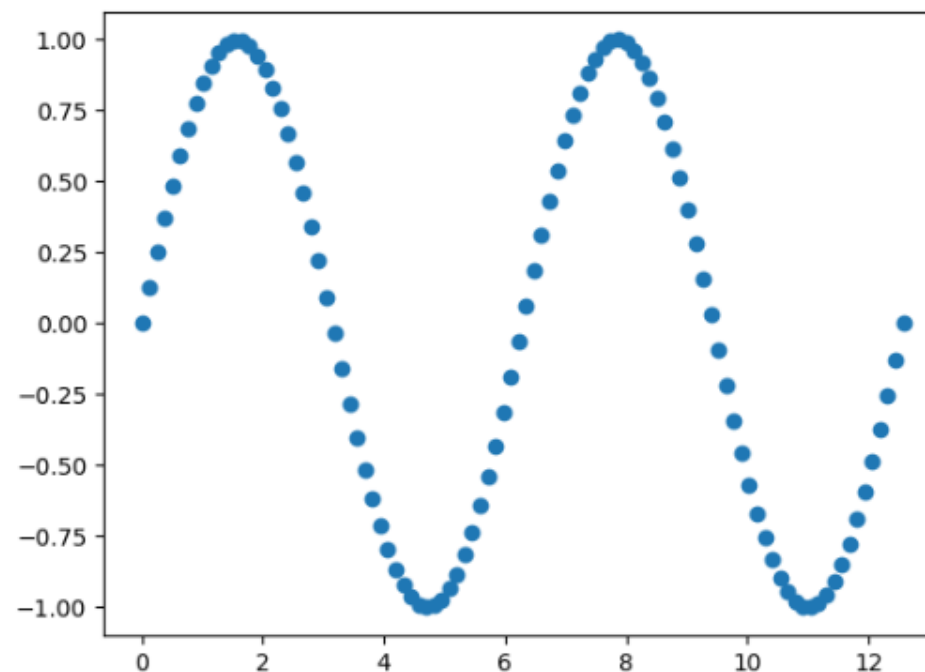
**Matplotlib** – это библиотека Python, предоставляющая API для встраивания графиков в приложения.

Гистограммы, столбцовые диаграммы, точечные диаграммы, круговые диаграммы - Matplotlib может отображать широкий спектр визуализаций. С Matplotlib можно создавать любые визуализации:

1. Линейные диаграммы;
2. Точечные диаграммы;
3. Диаграммы с областями;
4. Столбцовые диаграммы и гистограммы;
5. Круговые диаграммы;
6. Диаграммы «стебель-листья»;
- ...

## Пример применения библиотеки Matplotlib

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(0, 4*np.pi,100)
plt.figure()
plt.scatter(x,np.sin(x))
plt.show()
```



*Pandas* – это пакет Python с открытым исходным кодом, который предоставляет высокоэффективные, простые в использовании структуры данных и инструменты анализа для помеченных данных на языке программирования Python.

Pandas расшифровывается как *библиотека анализа данных Python*.

Pandas – это идеальный инструмент для обработки данных. Он предназначен для быстрой и простой обработки данных, чтения, агрегирования и визуализации.

Pandas берет данные в файле CSV или TSV или базу данных SQL и создает объект Python со строками и столбцами, который называется фреймом данных.

# Что делает *Pandas* фундаментальной библиотекой в изучении Python для Data Science?

1. Индексирование, манипулирование, переименование, сортировка, объединение фрейма данных;
2. Обновление, добавление, удаление столбцов из фрейма данных;
3. Восстановление недостающих файлов, обрабатывание недостающих данных;
4. Построение гистограммы.

Основная структура данных в *Pandas* – это датафрейм (DataFrame)



# Создание DataFrame в *Pandas*

🔍

✓  
0  
ОК.

{x}

✓  
0  
ОК.

📁

```
[1] import pandas as pd
```

```
df=pd.DataFrame([[ 'Anna', 23, 3],  
                  [ 'Sam', 36, 12],  
                  [ 'Bill', 33, 10],  
                  [ 'Moica', 25, 7],  
                  [ 'Lisa', 27, 7],  
                  [ 'Peter', 32, None]])
```

df

📄

|   | 0     | 1  | 2    |
|---|-------|----|------|
| 0 | Anna  | 23 | 3.0  |
| 1 | Sam   | 36 | 12.0 |
| 2 | Bill  | 33 | 10.0 |
| 3 | Moica | 25 | 7.0  |
| 4 | Lisa  | 27 | 7.0  |
| 5 | Peter | 32 | NaN  |

✎

Для удобства можно присваивать столбцам названия и вызывать при необходимости ячейку



Присвоим столбцам названия

{x}



0 сек.



```
df.columns=['name', 'age', 'opyt']  
df
```



|   | name  | age | opyt |
|---|-------|-----|------|
| 0 | Anna  | 23  | 3.0  |
| 1 | Sam   | 36  | 12.0 |
| 2 | Bill  | 33  | 10.0 |
| 3 | Moica | 25  | 7.0  |
| 4 | Lisa  | 27  | 7.0  |
| 5 | Peter | 32  | NaN  |



Вызов определенной ячейки



0 сек.



```
df.iloc[1,2]
```

12.0

Используя метод `iloc` можно вызывать определенные строки и столбцы фрейма

```
df.iloc[0:2,0]
```

|   |      |
|---|------|
| 0 | Anna |
| 1 | Sam  |

Name: name, dtype: object

В *Pandas* есть метод `describe()` для поиска описательной статистики

```
df.describe()
```

Описательная статистика

|       | age       | опыт      |
|-------|-----------|-----------|
| count | 6.000000  | 5.000000  |
| mean  | 29.333333 | 7.800000  |
| std   | 5.085928  | 3.420526  |
| min   | 23.000000 | 3.000000  |
| 25%   | 25.500000 | 7.000000  |
| 50%   | 29.500000 | 7.000000  |
| 75%   | 32.750000 | 10.000000 |
| max   | 36.000000 | 12.000000 |

Для вывода первых  
нескольких строк и  
последних есть  
соответствующие  
методы – head и tail

Вывод первых двух строк

[6] df.head(2)

|   | name | age | опыт |
|---|------|-----|------|
| 0 | Anna | 23  | 3.0  |
| 1 | Sam  | 36  | 12.0 |

Вывод последних двух строк

df.tail(2)

|   | name  | age | опыт |
|---|-------|-----|------|
| 4 | Lisa  | 27  | 7.0  |
| 5 | Peter | 32  | NaN  |

В *Pandas*  
достаточно  
удобно  
добавлять  
новые столбцы

{x}



✓  
0  
сек.



```
df['age_sq']=df['age']**2  
df['status']=5  
df
```



|   | name  | age | opyt | age_sq | status |
|---|-------|-----|------|--------|--------|
| 0 | Anna  | 23  | 3.0  | 529    | 5      |
| 1 | Sam   | 36  | 12.0 | 1296   | 5      |
| 2 | Bill  | 33  | 10.0 | 1089   | 5      |
| 3 | Moica | 25  | 7.0  | 625    | 5      |
| 4 | Lisa  | 27  | 7.0  | 729    | 5      |
| 5 | Peter | 32  | NaN  | 1024   | 5      |



<>

*NumPy* – один из самых фундаментальных пакетов в Python – универсальный пакет для обработки массивов. Он предоставляет высокопроизводительные объекты многомерных массивов и инструменты для работы с массивами. NumPy – это эффективный контейнер универсальных многомерных данных.

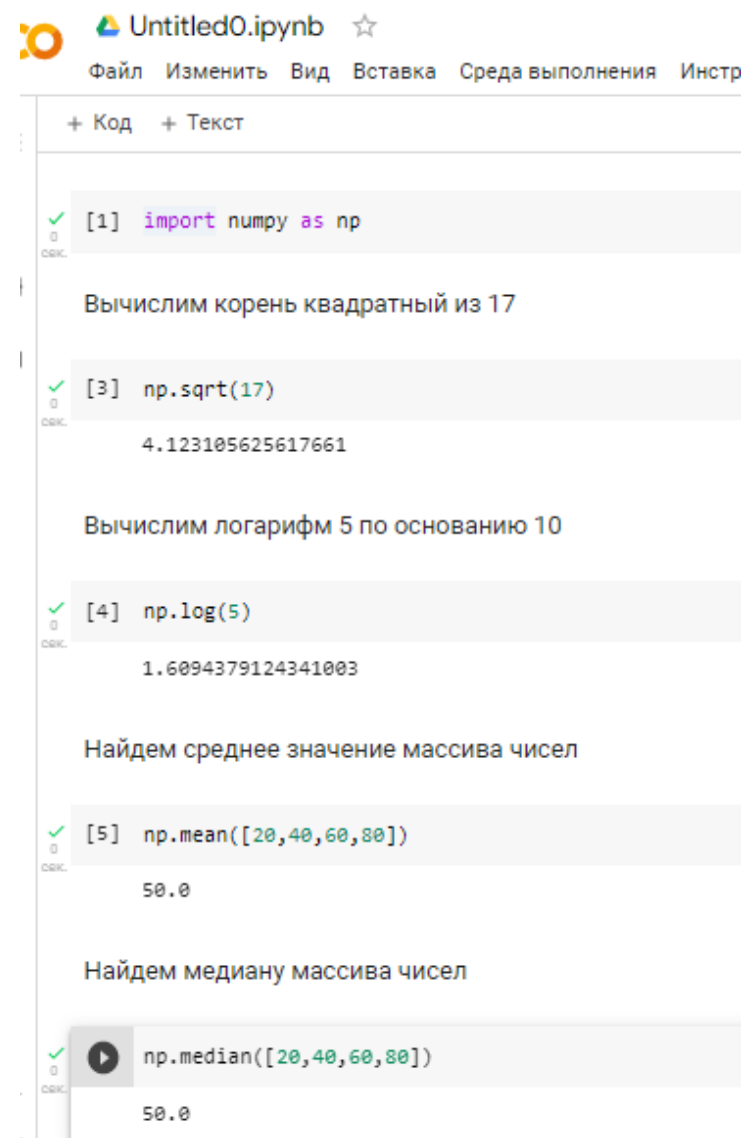
Основной объект NumPy – это однородный многомерный массив. Это таблица элементов или чисел одного и того же типа данных, проиндексированная набором натуральных чисел. В NumPy размеры называются *осями*, а число осей называется *рангом*. Класс массива NumPy называется *array*.

NumPy используется для обработки массивов, в которых хранятся значения одного и того же типа данных. NumPy облегчает математические операции над массивами. Это значительно повышает производительность и, соответственно, ускоряет время выполнения.

# Перечислим основные действия, которые ВОЗМОЖНО ВЫПОЛНИТЬ в рамках NumPy:

1. Основные операции с массивами: добавление, умножение, срез, выравнивание, изменение формы, индексирование массивов;
2. Расширенные операции с массивами: стековые массивы, разбиение на секции, широковещательные массивы;
3. Работа с DateTime или линейной алгеброй;
4. Основные нарезки и расширенное индексирование в NumPy Python.

# Пример работы с функциями библиотеки NumPy



The screenshot displays a Jupyter Notebook titled "Untitled0.ipynb". The interface includes a menu bar with options: "Файл", "Изменить", "Вид", "Вставка", "Среда выполнения", and "Инстр". Below the menu, there are tabs for "+ Код" and "+ Текст". The notebook contains five code cells, each with a green checkmark and a "0" icon, indicating successful execution. The first cell imports NumPy as 'np'. The second cell calculates the square root of 17 using 'np.sqrt(17)'. The third cell calculates the logarithm of 5 to the base 10 using 'np.log(5)'. The fourth cell calculates the mean of the array [20, 40, 60, 80] using 'np.mean([20, 40, 60, 80])'. The fifth cell calculates the median of the same array using 'np.median([20, 40, 60, 80])'. Each code cell is followed by a text cell describing the operation.

```
[1] import numpy as np
```

Вычислим корень квадратный из 17

```
[3] np.sqrt(17)
```

4.123105625617661

Вычислим логарифм 5 по основанию 10

```
[4] np.log(5)
```

1.6094379124341003

Найдем среднее значение массива чисел

```
[5] np.mean([20, 40, 60, 80])
```

50.0

Найдем медиану массива чисел

```
np.median([20, 40, 60, 80])
```

50.0



# Пример работы с массивами библиотеки

Представим в виде массива список

```
[ ] np.array([2,3,4])  
array([2, 3, 4])
```

Создадим двумерный массив

```
np.array([[2,3,4],[5,6,7]])  
array([[2, 3, 4],  
       [5, 6, 7]])
```

Умножим все значения массива на 100

```
[ ] t=np.array([4,5,6])  
t*100  
array([400, 500, 600])
```

Возведем в квадрат элементы массива

```
[ ] np.array([2,3,4])**2  
array([ 4,  9, 16])
```

Узнаем форму нашего массива

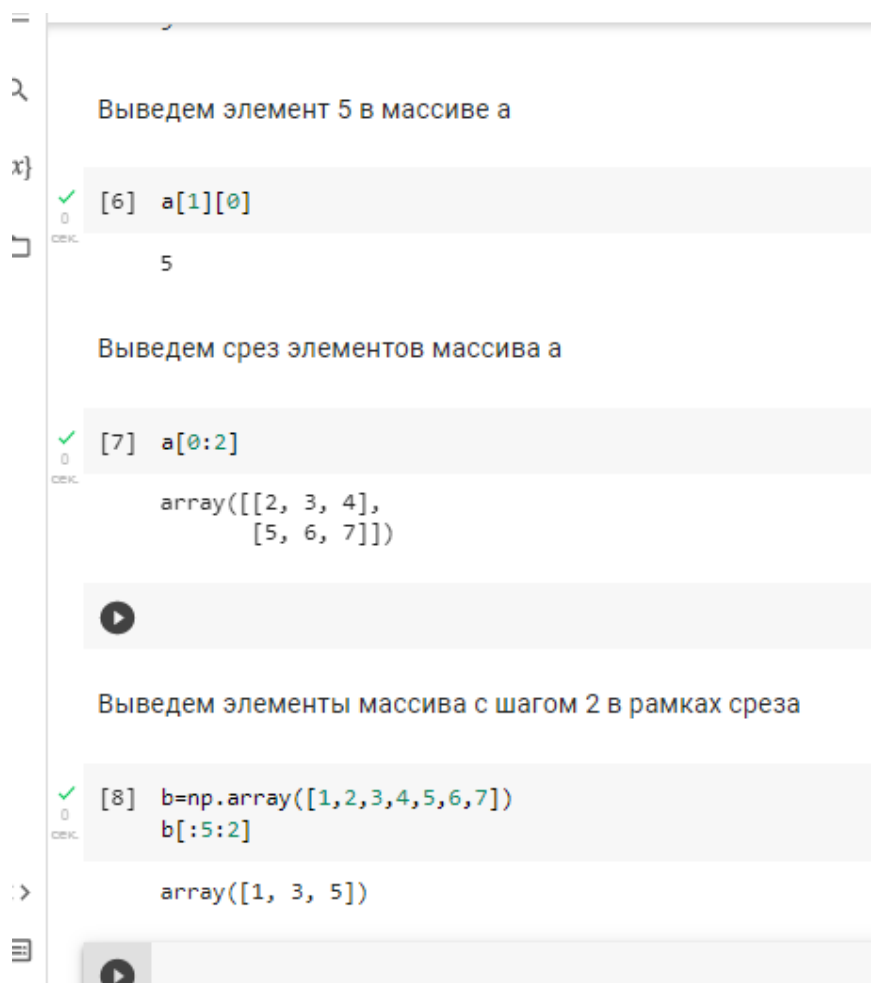
```
a=np.array([[2,3,4],[5,6,7],[7,8,9]])  
a.shape  
(3, 3)
```

Узнаем общее число элементов

```
[ ] a.size  
9
```

Выведем элемент 5 в массиве a

# Пример работы с массивами библиотеки



```
Выведем элемент 5 в массиве a

[6] a[1][0]
5

Выведем срез элементов массива a

[7] a[0:2]
array([[2, 3, 4],
       [5, 6, 7]])

Выведем элементы массива с шагом 2 в рамках среза

[8] b=np.array([1,2,3,4,5,6,7])
    b[:5:2]
array([1, 3, 5])
```

# Создание массивов из единиц и нулей с помощью библиотеки NumPy

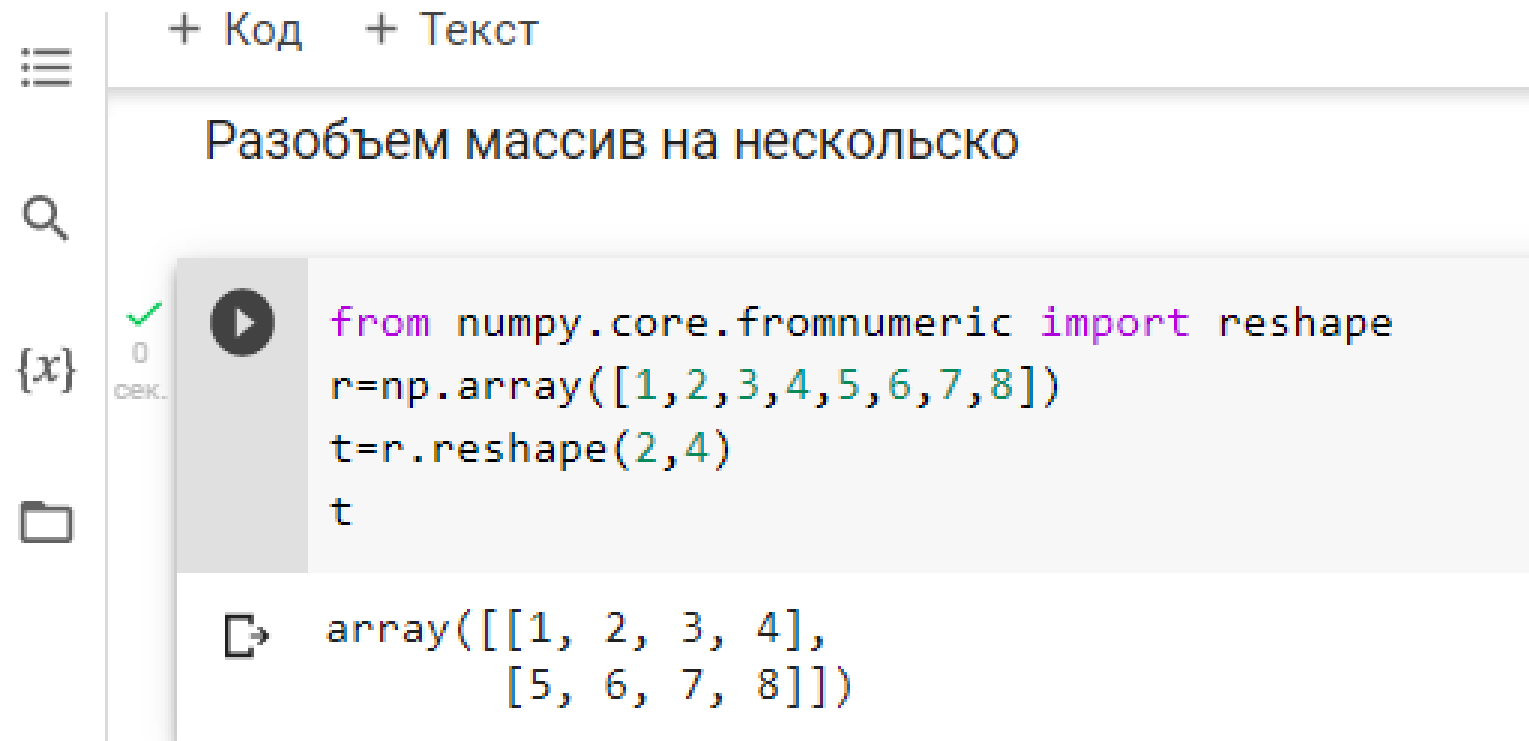
Создание массива из единиц

```
✓ [13] q=np.ones((2,4))  
0 сек.  
q  
  
array([[1., 1., 1., 1.],  
       [1., 1., 1., 1.]])
```

Создание массива из нулей

```
✓ [13] w=np.zeros((2,3))  
0 сек.  
w  
  
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

# Разбиение массива на несколько с помощью библиотеки NumPy



The screenshot shows a Jupyter Notebook interface. On the left is a sidebar with icons for a menu, search, a variable  $\{x\}$ , and a file folder. The top of the notebook area has tabs for '+ Код' and '+ Текст'. The main cell is titled 'Разобьем массив на несколько'. It contains a code cell with the following Python code:

```
from numpy.core.fromnumeric import reshape
r=np.array([1,2,3,4,5,6,7,8])
t=r.reshape(2,4)
t
```

Below the code cell, the output is displayed as a 2x4 array:

```
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
```

## Создание массива с определенными характеристиками, округление значений его, добавление к элементам массива значений с помощью библиотеки NumPy

Создание массива от 1 до 10 с шагом 0,1

```
[5] M=np.arange(1,10,0.1)
```

M

```
array([1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2,
       2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5,
       3.6, 3.7, 3.8, 3.9, 4. , 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8,
       4.9, 5. , 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6. , 6.1,
       6.2, 6.3, 6.4, 6.5, 6.6, 6.7, 6.8, 6.9, 7. , 7.1, 7.2, 7.3, 7.4,
       7.5, 7.6, 7.7, 7.8, 7.9, 8. , 8.1, 8.2, 8.3, 8.4, 8.5, 8.6, 8.7,
       8.8, 8.9, 9. , 9.1, 9.2, 9.3, 9.4, 9.5, 9.6, 9.7, 9.8, 9.9])
```

Округлим значения массива до двух знаков после запятой

```
▶ M.round(2)
```

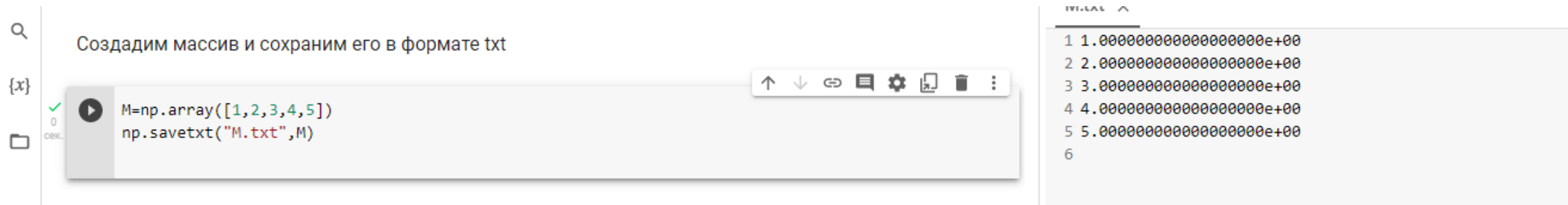
```
array([1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2,
       2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5,
       3.6, 3.7, 3.8, 3.9, 4. , 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8,
       4.9, 5. , 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6. , 6.1,
       6.2, 6.3, 6.4, 6.5, 6.6, 6.7, 6.8, 6.9, 7. , 7.1, 7.2, 7.3, 7.4,
       7.5, 7.6, 7.7, 7.8, 7.9, 8. , 8.1, 8.2, 8.3, 8.4, 8.5, 8.6, 8.7,
       8.8, 8.9, 9. , 9.1, 9.2, 9.3, 9.4, 9.5, 9.6, 9.7, 9.8, 9.9])
```

Добавим к каждому элементу массива значение 2

```
▶ np.add(M,2)
```

```
array([ 3. ,  3.1,  3.2,  3.3,  3.4,  3.5,  3.6,  3.7,  3.8,  3.9,  4. ,
        4.1,  4.2,  4.3,  4.4,  4.5,  4.6,  4.7,  4.8,  4.9,  5. ,  5.1,
        5.2,  5.3,  5.4,  5.5,  5.6,  5.7,  5.8,  5.9,  6. ,  6.1,  6.2,
        6.3,  6.4,  6.5,  6.6,  6.7,  6.8,  6.9,  7. ,  7.1,  7.2,  7.3,
        7.4,  7.5,  7.6,  7.7,  7.8,  7.9,  8. ,  8.1,  8.2,  8.3,  8.4,
        8.5,  8.6,  8.7,  8.8,  8.9,  9. ,  9.1,  9.2,  9.3,  9.4,  9.5,
        9.6,  9.7,  9.8,  9.9, 10. , 10.1, 10.2, 10.3, 10.4, 10.5, 10.6,
       10.7, 10.8, 10.9, 11. , 11.1, 11.2, 11.3, 11.4, 11.5, 11.6, 11.7,
       11.8, 11.9])
```

# Создание массива и сохранение его отдельным файлом



The image shows a Jupyter Notebook interface. On the left, a sidebar contains a search icon, a variable `{x}`, and a file explorer icon. The main area has a text prompt "Создадим массив и сохраним его в формате txt" and a code cell. The code cell contains the following Python code:

```
M=np.array([1,2,3,4,5])  
np.savetxt("M.txt",M)
```

Below the code cell, a status bar shows a green checkmark, a play button, and the text "0 сек.". To the right of the code cell is a toolbar with icons for up, down, link, comment, settings, copy, delete, and a menu. The output area on the right displays the following text:

```
1 1.0000000000000000e+00  
2 2.0000000000000000e+00  
3 3.0000000000000000e+00  
4 4.0000000000000000e+00  
5 5.0000000000000000e+00  
6
```

# Seaborn - это расширение Matplotlib с ДОПОЛНИТЕЛЬНЫМИ ВОЗМОЖНОСТЯМИ.

Matplotlib используется для основного построения столбцовых, круговых, линейных, точечных диаграмм и пр., в то время как Seaborn предоставляет множество шаблонов визуализации с меньшим количеством синтаксических правил, причем более простых.

С помощью данной библиотеки можно:

1. Определять отношения между несколькими переменными (корреляция);
2. Соблюдать качественные переменные для агрегированных статистических данных;
3. Анализировать одномерные или двумерные распределения и сравнивать их между различными подмножествами данных;
4. Построить модели линейной регрессии для зависимых переменных.